

Document number: P1978R0
Authors: Andrzej Krzemiński
Glen Joseph Fernandes
Nevin Liber
Peter Dimov
Date: 2019-11-15
Audience: Library Evolution Working Group
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>
Glen Joseph Fernandes <glenjofe at gmail dot com>

Rename `_default_init` functions and do nothing more

1. Introduction

While addressing national body comment [DE002](#), LEWG came up with the solution that went beyond changing function names (as requested in DE002), but also changed the design of these functions by applying constraints which renders these functions very difficult to use in generic contexts. In this paper we propose a smaller solution that is more in line with DE002, and that does not attempt to do a last-minute redesign.

2. Discussion

The problem reported in DE002 is that functions that are meant to perform default initialization of heap-allocated objects are spelled like `make_unique_default_init`, which may mislead non-expert programmers — who do not understand terms from the Standard like "default initialization" — to believe that objects will be initialized to their safe default values (which experts call "value initialization").

In response to that, [P1973R0](#) changes the names to `make_unique_uninitialized`, which now confuses a different group of non-experts, by incorrectly implying that these objects are uninitialized. In order to avoid this new confusion, P1973R0 additionally proposes SFINAE-like constraints that make these functions available only for types that are trivially default-constructible, which only partially mitigates the new confusion, but also makes these functions very difficult to use inside templates.

Here is an example that now becomes difficult to write:

```
template <typename T, size_t Size>
std::unique_ptr<T[]> initialize_array()
{
    auto result = std::make_unique_default_init<T[]>(Size);
    for (size_t i = 0; i != Size; ++i)
        result[i] = produce_value<T>(i);

    return result;
}
```

We first need to allocate the array, and only then fill it with meaningful values, so the initial values are never read. We want to perform this initialization as fast as we can for every τ . This means that for types with user-provided default constructor we have to call this constructor, but for trivial types we do the trivial initialization. With the solution proposed in P1973R0, writing this template is not that easy anymore. We have to either provide a specialisation or use an `if constexpr` to distinguish the two cases. The solution in P1973R0 seems to be missing the point of introducing these functions in the first place, and disregarding this idiomatic use. This, in turn, impedes the migration from `boost::make_unique_noinit`.

In the light of the above, we propose, as per DE002, to only change the names of these functions to `make_unique_minimal_init`, and refrain from any other design changes. Term "minimal" seems to be conveying the

idea behind the idiom on the one hand, and on the other not to overlap with any expert term in the Standard, thus avoiding any confusion.

3. Proposed wording

Changes are relative to N4835.

Replace all occurrences of ~~make_unique_default_init~~ with make_unique_minimal_init.

Replace all occurrences of ~~make_shared_default_init~~ with make_shared_minimal_init.

Replace all occurrences of ~~allocate_shared_default_init~~ with allocate_shared_minimal_init.

4. References

1. [DE002], "Rename make_unique/shared_default_init to make_unique/shared_nonvalue_init", (DE002, <https://github.com/cplusplus/nbballot/issues/2>)
2. [P1973r0], Nicolai Josuttis, "Rename '_default_init' Functions", (P1973R0, http://wiki.edg.com/pub/Wg21belfast/LibraryEvolutionWorkingGroup/D1973R0_rename_default_init_funcs.pdf)